08/10/2026

Mini Proyecto MIPS

Briana Chang

Jose Gabela

UCOM250 - Organización y Arquitectura de Computadores

2026

Contenido

Descripción 3

Análisis 3

Implementación 4

Conclusiones 7

Bibliografía

Descripción

Una fábrica utiliza una máquina que produce 25 piezas por hora. La máquina trabaja durante 8 horas. El grupo debe calcular cuántas piezas se producen en total y comparar la tasa de producción por hora con el número de horas trabajadas.

Análisis

Datos y operaciones del programa

Dato / resultado	Valor / propósito	Operación requerida	Instrucción MIPS
piezas	25; piezas producidas por hora	Cargar desde memoria	lw
horas	8; horas trabajadas	Cargar desde memoria	lw
produccionTotal	200; total producido	Multiplicar piezas por horas	mul
comparacion	1 si son diferentes; 0 si son iguales	Comparar y controlar el flujo	beq + li
resultado final	Texto y total mostrados al usuario	Imprimir cadena y entero	syscall
resultados en memoria	Guardar total y bandera	Almacenar en memoria	sw

Flujo de datos

Los valores se definen en la sección .data. La instrucción lw los carga en $t0 y $t1; la CPU calcula la producción en $t2. beq compara los operandos y decide si salta al final. Si no son iguales, el programa asigna 1 a $t3 y lo almacena. Finalmente, imprime el mensaje y el valor calculado.

Implementación

Versión base

El punto de partida define datos y resultados en memoria y propone cargar dos operandos, procesarlos y almacenar resultados. En el primer avance se sustituyeron los valores iniciales por 25 y 8 y se calcularon producto, comparación, restas, división, residuo e igualdad. En el Avance 2 se precisó el comportamiento condicional de salida.

Versión final: programa_final.s

```
.data
piezas: .word 25 # Piezas producidas por hora
horas: .word 8 # Horas trabajadas
produccionTotal:.word 0 # Resultado de piezas x horas
comparacion: .word 0 # 1 si piezas y horas son distintas
msg_produccion: .asciiz "Produccion calculada: "

.text
.globl main
main:
# Cargar los datos desde memoria a registros
lw $t0, piezas
lw $t1, horas

# Calcular la produccion y guardarla en memoria
mul $t2, $t0, $t1
sw $t2, produccionTotal

# Si piezas y horas son iguales, no imprimir el mensaje
beq $t0, $t1, fin

# Son diferentes: registrar 1 y mostrar mensaje y total
li $t3, 1
sw $t3, comparacion
li $v0, 4
la $a0, msg_produccion
syscall
li $v0, 1
move $a0, $t2
syscall

fin:
li $v0, 10
syscall
```

Descripción del código

El programa carga las dos entradas con lw y calcula el producto con mul. Guarda el total con sw. El salto beq termina la ejecución sin imprimir cuando los valores son iguales. Cuando son distintos, se guarda 1 en comparación y se imprime el mensaje seguido del entero. Se evita almacenar un valor no inicializado en la variable de comparación.

Resultados esperados

Ubicación	Valor esperado	Significado
$t0 / piezas	25	Piezas por hora
$t1 / horas	8	Horas trabajadas
$t2 / produccionTotal	200	Producción del día
$t3 / comparacion	1	25 ≠ 8
Pantalla	Producción calculada: 200	Salida condicional solicitada

Archivo	**Qué debe mostrar**
evidencias/codigo.png	El código cargado en el simulador, incluyendo datos, cálculo y salto condicional.
evidencias/registros.png	Los registros $t0 = 25, $t1 = 8, $t2 = 200 y $t3 = 1 luego de la ejecución.
evidencias/resultado.png	La consola con la salida “Producción calculada: 200”.

Conclusiones

La actividad permitió relacionar una situación concreta con una secuencia de instrucciones de bajo nivel. Al seguir los datos desde la memoria hasta los registros y de vuelta a memoria, comprendimos mejor el papel de cada etapa en la ejecución.

La parte que requirió más cuidado fue traducir la condición del enunciado a un flujo correcto: el programa solo debía imprimir el resultado cuando piezas por hora y horas trabajadas fueran diferentes. Se resolvió comparando ambos registros con un salto condicional y revisando el orden de inicialización y almacenamiento de los datos.

La implementación práctica ayudó a comprender que una comparación puede cambiar el recorrido del programa y que cada resultado debe guardarse explícitamente si se necesita consultar después. En una nueva resolución, validaremos desde el inicio los valores esperados en registros, memoria y consola, y mantendremos separadas las capturas de evidencia de las explicaciones.

Bibliografía

MIPS Assembler and Runtime Simulator (n.d.). MARS. http://courses.missouristate.edu/kenvollmar/mars/

MIPS Instruction Set (n.d.). University of New South Wales. Recuperado el 08 de octubre de 2026 de https://cgi.cse.unsw.edu.au/~cs1521/current/resources/mips-guide.html

Patterson, D. A., & Hennessy, J. L. (2021). Computer organization and design: The hardware/software interface (6th ed.). Morgan Kaufmann.